SDC 2026 · XMUM AI CLUB · PHASE 1 PRELIMINARY

CollectLoop

Project Proposal · RREF COLLECTION · Submission deadline 25 October 2026

1 Team and Project Overview

FIELD	RESPONSE
Team Name	RREF COLLECTION
Selected Domain	Domain 2 — Orbyte Smart SME Challenge
Selected SDG Track (1-17)	SDG 8 — Decent Work and Economic Growth (primary); SDG 9 — Industry, Innovation and Infrastructure (secondary)
Project / Solution Name	CollectLoop
Name (Leader)	XIE KAIRUI
Email & Phone (Leader)	2173083338@qq.com
University Affiliation(s)	Xiamen University Malaysia
Number of Team Members	4

In one sentence. CollectLoop chases the promise, not the invoice: it keeps every verbal payment commitment on record, drafts the reminder only when the promise is gone, and stays silent - on purpose - while it is still good.

2 Executive Summary & Problem Statement

2.1. The Problem Context

Describe the specific, real-world challenge your team is tackling. Who is affected, and why does it matter? [Maximum 250 words]

A micro-wholesaler in Klang issues 60 to 80 invoices a month. Roughly one in six is paid late. Chasing those invoices is nobody's job description: it belongs to the owner, squeezed between deliveries and stock checks, in whatever five minutes are free.

Each reminder costs about twelve minutes - find the invoice, check whether it was already paid, decide how firm to be this time, write it, verify the numbers, file it somewhere.

Because it is done in fragments, three failures follow. Tone drifts, so a customer three days late receives the same blunt message as one sixty days late, and goodwill is spent for nothing. Nothing is recorded, so when the owner is travelling nobody knows who was contacted, how firmly, or what was agreed. And it is the first task dropped on a busy day, which pushes cash further out - the one cost a micro-enterprise cannot absorb.

The costliest failure is the invisible one: the promise is never written down. Customers already name payment dates on calls and in WhatsApp threads. Because those commitments live nowhere, the owner either chases a customer

who has already promised, spending goodwill on someone who is not the problem, or forgets the one who broke a date they chose themselves. Neither is a writing problem; both are a memory problem, and no reminder template fixes a memory problem.

This affects more than a million micro, small and medium enterprises in Malaysia, and the staff whose salaries depend on receivables arriving on time.

2.2. The Proposed AI Solution

High-level overview: how does the application apply AI to the problem above, and which SDC Phase 1 workshop tools does it leverage? [Maximum 300 words]

CollectLoop is a collections assistant whose unit of work is the promise, not the invoice. Every customer carries a small ledger: what was promised, for which date, through which channel, by whom, and whether it was kept. Everything downstream is decided by that ledger, in deterministic, unit-tested rules. No promise on record: draft the first message in the tone the ageing bucket calls for. Promise still open: draft nothing, hold the invoice, and count the hold as a real outcome. Promise broken: draft, and quote the customer's own date back at them. Two breaches escalate the tone and multiply queue priority.

AI is used at exactly one point, deliberately: writing the prose. A language model receives a restricted fact set - invoice number, amount, due date, days overdue, the promise verbatim - and returns the body of one email. It does not choose who is chased, does not set the tone, and cannot send. A person opens each draft in an editable box and approves or skips it; both outcomes are written to an append-only log with a hash chain. If no model is configured, or the endpoint fails, the system falls back to a hand-written template and labels the draft source in the interface, so the product degrades instead of white-screening.

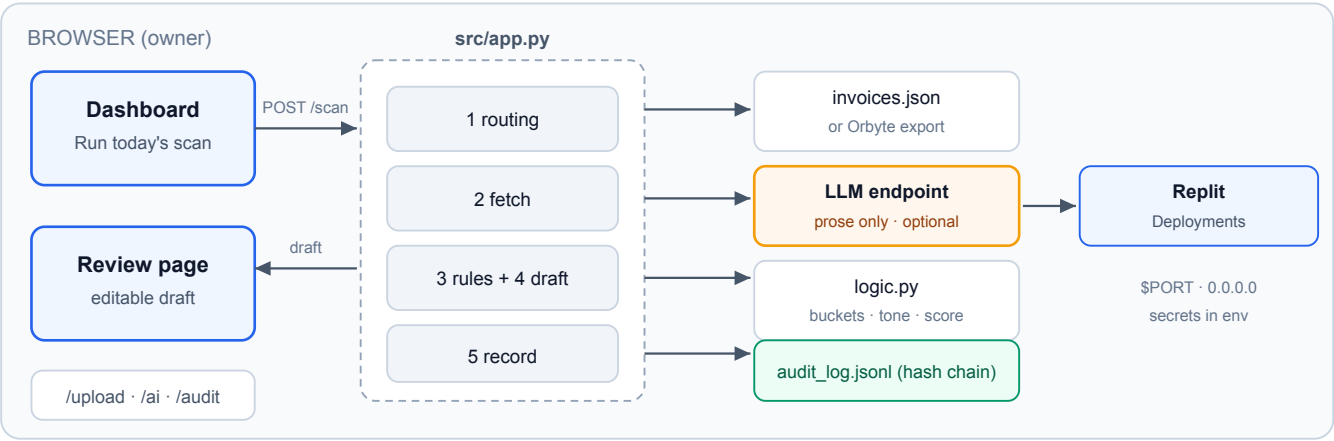
The Phase 1 workshops shaped three decisions. The GenAI API session set the model layer: a system prompt with hard prohibitions, and an explicit fallback path. The database-architecture session shaped the audit design, because a collections trail has to be tamper-evident, not merely persisted. Cloud deployment is how this reaches judges as a public URL rather than a screenshot. The stack is the Python standard library only, because the clearest lesson was that a prototype dies at the dependency step far more often than at the logic step.

3 Technical Architecture & Implementation Plan

3.1. Conceptual Tech Stack

Technologies, frameworks, APIs and databases used to build a functional, deployed prototype.

LAYER	TECHNOLOGY	ROLE
Language	Python 3.11	Backend, routing and all business logic
Web layer	Python standard library `http.server`	Server-rendered HTML; no framework, no build step
Templates	Plain HTML + one CSS file	Six pages; no client-side JavaScript at all
Persistence	JSON files on disk + append-only JSONL audit log	Invoice book, runtime state, hash-chained decision log
AI layer	Any OpenAI-compatible chat completions endpoint	Drafts one email body from a restricted fact set
AI fallback	Deterministic template in `src/llm.py`	Keeps the product working with no model configured
Integration	Orbyte export adapter (`src/orbyte.py`)	Orbyte field names become the system of record; replay mode by default
Testing	`unittest`, 45 tests, zero dependencies	Decision rules, intake validation, security headers
Hosting	Replit Deployments (sponsor-provided Core)	Public live URL for judges
Version control	Git + GitHub, Apache-2.0	Public repository, full commit history



Official SDC sponsor GenAI API or Cloud Platform? Yes - **Replit Core**, issued to the team leader under the sponsor agreement. It hosts the deployed prototype and the public live URL judges open. **Orbyte** (Domain 2 sponsor) supplied the domain brief and data contract; no Orbyte API credentials were issued, so the adapter runs on a synthetic export and no

Orbyte endpoint has been called. No sponsor GenAI credit was used. **RunPod: not used** - there is no GPU workload in this prototype.

3.2. Design-Thinking & Deployment Strategy

How the team applied design thinking to make the solution viable, usable and deployable by Phase 2. [Maximum 200 words]

Design thinking was applied as narrowing decisions, not as a phase label.

Empathise: we interviewed a micro-wholesaler's owner about the last time a customer paid late. The detail that changed the project was not the invoice - it was that three customers had already named a date, on a call or in a chat, and that nobody could recall which.

Define: the problem statement became "the promise exists but never reaches the system", not "reminders are slow to write". The second framing invites a template; the first one does not.

Ideate: we built the obvious version first - ask a model how urgent each customer is. It gave different answers on identical inputs. Killing it and putting urgency in an ageing table made the product both better and explainable to a judge in one sentence.

Prototype and test: rules first, model last. Forty-five unit tests cover the decision layer and the intake validation, so tone and priority can be changed without a regression.

Deployment strategy: zero dependencies, one process, all configuration through environment variables, and the rules in one file, so a second SME reuses the product by editing config rather than code - which is what "Orbyte fit and reuse" requires.

4 Preliminary Environmental & SDG Impact Framework

Baseline and target for judges to assess Innovation, Creativity and Feasibility. Numbers are estimated or cited where measurement is not yet possible.

METRIC CATEGORY	TARGET SDG SUB-TARGET(S)	CURRENT PROBLEM BASELINE	CORE MEASURABLE KPI	PHASE 2 IMPACT TARGET
Receivables cycle (primary)	**SDG 8.3** - promote development-oriented policies that support micro-, small- and medium-sized enterprises.	12 minutes of manual work per reminder; roughly 12 late invoices a month, so about 2.4 hours of the owner's month, done in fragments and often skipped. Illustrative DSO on our synthetic book: 34 days.	Minutes of human work per reminder; and share of would-be chases correctly suppressed because a promise is still open.	12 min to 3 min per reminder (-75%), with at least 30% of the scanned queue held instead of chased.
Productivity (primary)	**SDG 8.2** - achieve higher levels of economic productivity through technological upgrading.	Every reminder is typed from scratch, at 12 minutes each, by the person who should be selling. Zero reminders are logged, so no process can be improved later.	Minutes recovered per month per SME; share of decisions that reach the append-only audit log.	100% of approved or skipped drafts logged with a hash chain; 2.4 hours a month recovered per SME.
Access to finance (secondary)	**SDG 9.3** - increase the access of small-scale enterprises to credit.	No receivables history exists in a form a lender would accept, so working-capital credit is priced on the owner's word.	Count of exportable, tamper-evident payment-promise records per customer.	One exportable per-customer promise history that a lender can audit, covering 100% of invoiced customers.

On the honesty of these numbers. The 12-minute baseline comes from our own interview notes and is stated as an assumption, not a measurement; the 60 to 80 invoice volume and the one-in-six late rate are the owner's estimates. The 34-day DSO figure is illustrative on synthetic data. Phase 2 measures the real values on the same book, so the KPI in the third column is what we will report rather than a projection we cannot support. The MSME population figure is from the Department of Statistics Malaysia, which reports that MSMEs make up 97.4% of business establishments in Malaysia.

What we deliberately do not claim. No SDG 12 or 13 claim: removing paper reminders is real but too small to measure, and claiming it would be a label rather than an outcome. No SDG 10 claim: helping small businesses is not the same as reducing inequality between groups of people.

5 Eligibility & Readiness Checklist

☑ All team members have completed the mandatory GenAI, Database, and Cloud Deployment upskilling workshops.

☑ The project concept addresses a verified real-world problem mapped directly to one or more UN SDGs.

☑ The proposed solution targets a functional, deployed application rather than a theoretical business case study.

☑ All fields in this template have been completed within the word limits stated.

6 IP & Originality Declaration

6.1. Pre-Existing / Prior-Owned Work

Does this submission include code, design, research or other work created before the start of this hackathon?

Answer	**No.** All application code, templates, copy, the decision rules and this document were created during the SDC 2026 hackathon period.

6.2. Third-Party Code, Datasets, or Models

Does the project use third-party code libraries, frameworks, datasets or pre-trained models that the team did not create?

NAME	TYPE	LICENSE	NOTES
Python 3.11 standard library	Code	PSF-2.0	The only runtime dependency; used for HTTP, JSON, hashing and escaping. No pip install step.
`gpt-4o-mini` class chat model (optional, via OpenAI-compatible API)	Model	Provider terms of service	Pre-trained, called as a hosted API; not redistributed, not fine-tuned.
Synthetic invoice book and Orbyte export sample	Dataset	Self-authored, synthetic	Invented for this project; no real personal data.

6.3. Employment-Related IP Conflicts

Is any team member bound by an agreement under which an employer could claim ownership of work created during this hackathon?

Answer	**No.** All members are students and are not bound by any such employment, internship or consulting agreement.

7 Declaration

By submitting this proposal, the team leader confirms that the information provided is accurate to the best of the team's knowledge and that the project is original work created during the SDC 2026 hackathon period.

Signature - XIE KAIRUI, Team Leader, RREF COLLECTION

Date - 23 October 2026